JWT (JSON WEB TOKEN)

---

json web token is a compact token that carries claims about a user and is signed by the server

1. user login in with username and password
2. server verifies credentials
3. server issues JWT
4. client sends JWT in future requests
5. server verifies JWT and returns response

JWT HAS THREE PARTS

1.Header

Defines the token type and the algorithm used to sign the token

Example:

```
{
 "typ": "JWT",
 "alg": "HS256"
}
```

Note:

1. typ: Token type (aways JWT)
2. alg: Algorithm used to sign
   (e.g. HS256, RS256)

2.Payload

contains the claims (data) about the user and other information

Example:

```
{
 "userID": "12345",
 "role": "student",
 "exp": 1764546790"
}
```

Note:

1. userID: unique user identifier
2. role: user role (eg student)
3. exp: Expiration time (unix timestamp)
4. other claims as needed

3.Signature

verifies the tokens integrity and authenticity using the header payload and a secret (or private key)

Example:

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret
)
```

Note:

1. Created using the header and payload
2. signed with a secret (or private key)
3. Ensures the token is not altered and is from a trusted issuer

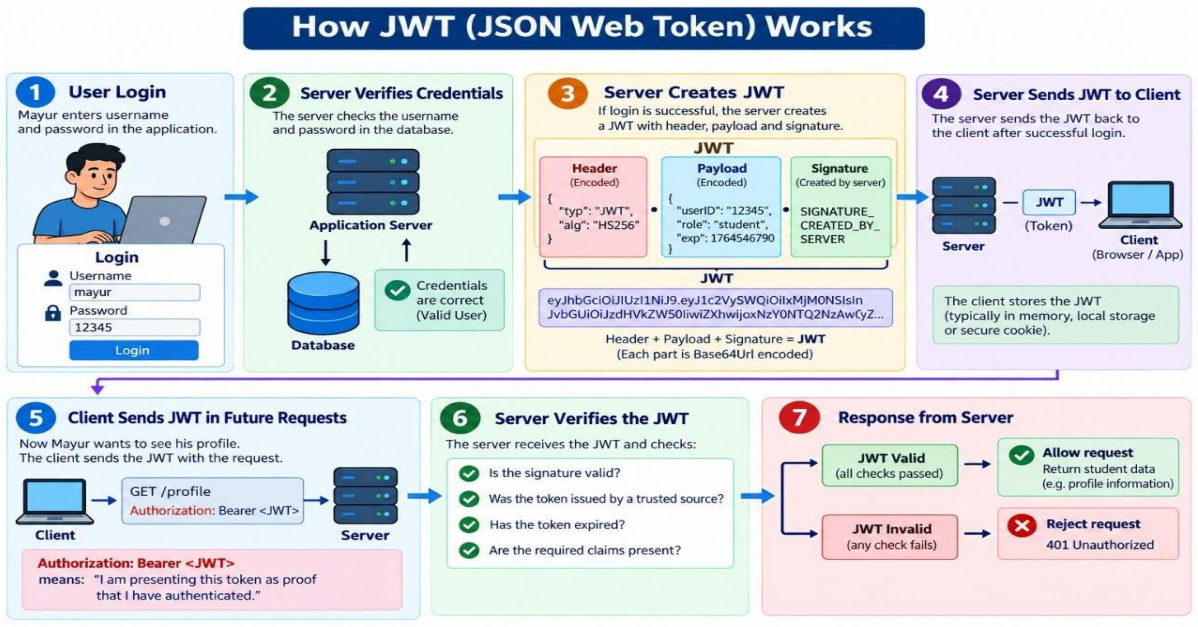
---

PROS

1. Good for APIs
2. Useful in microservices
3. Token carries claims
4. Scalable for distributed systems

CONS

1. harder revocation
2. stolen bearer token risk
3. claims can become stale
4. token can be larger



IF USER LOGIN INTO THE SERVER

1.suppose mayur logs in

1. username: mayur
2. password: 12345

2.the server creates

header

```
{
  "typ": "JWT",
  "alg": "HS256"
}
```

Payload

```
{
  "userID": "12345",
  "role": "student",
  "exp": 1764546790
}
```

Signature

SIGNATURE_CREATED_BY_SERVER

Header+Payload+Signature = JWT

WHAT HAPPENS AFTER LOGIN

1. The server sends the JWT to the client
   server -> JWT -> client
2. Now suppose the students want to see their profile
   The client sends:
   1. GET /profile
   2. authorization: Bearer <JWT>

Note: What is authorization: Bearer <JWT>
"I am presenting this token as proof that I have authenticated."

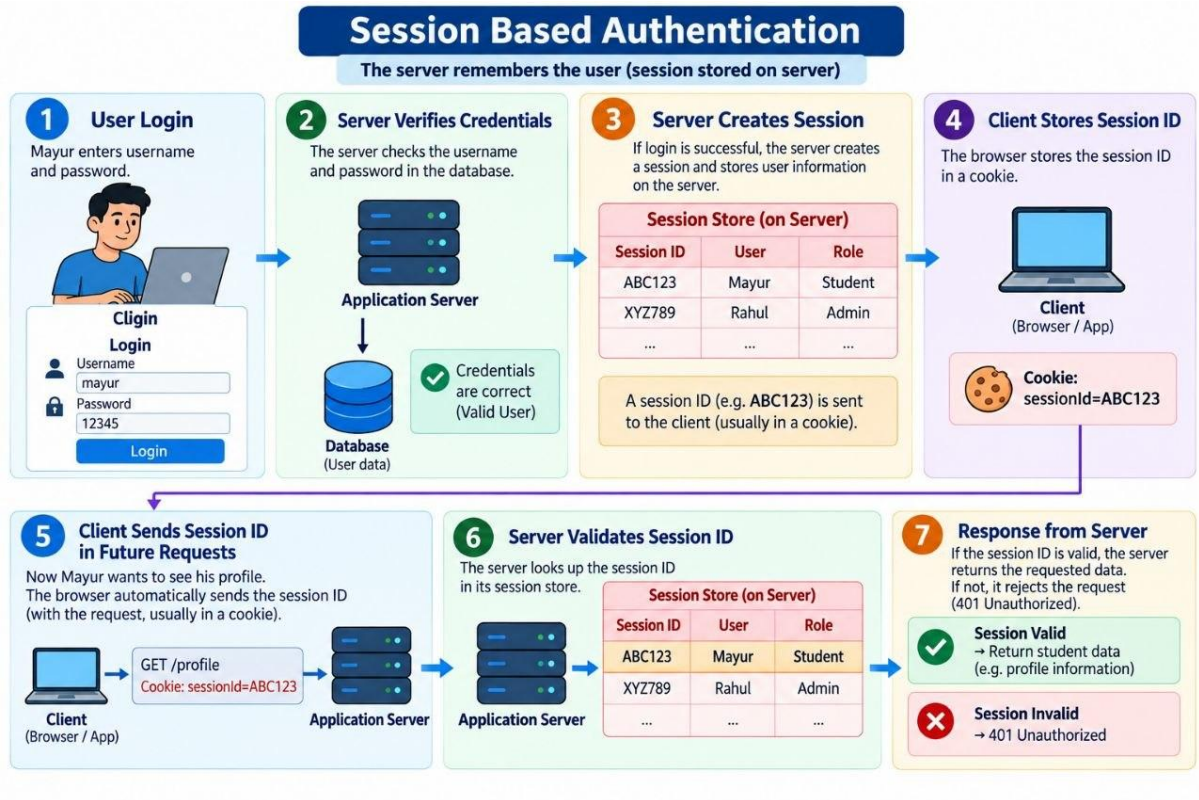
3. Server Verifies The JWT
   the server receives:
   client -> JWT -> server

4. server checks things such as:
   1. Is the signature valid?
   2. Was the token issued by a trusted source?
   3. Has the token expired?
   4. Are the required claims present?

5. If everything is okay
    1. JWT Valid -> allow request -> return student data

6. If something wrong
    1. JWT Invalid -> Reject request -> 401 unauthorized

Session-Based Authentication



SESSION AUTHENTICATION MEANS:

1. after login, the server creates a session and remembers the user on the server.
2. The browser does not carry all the user information. It mainly carries a Session ID.

WHEN USER LOGIN

1. User Enters
    1. Username: mayur
    2. Password: ABC123

The browser sends this login information to the application server.
User -----> Application server

SERVER VERIFIES CREDENTIALS

The application server checks whether the username and password are correct.
Application server ---> database ----> Are credentials correct -----> Valid user

SERVER CREATES A SESSION

After successful login, the server creates a session ID

1. Session ID = ABC123

The server stores information about that session:
Session Store

1. ABC123 Mayur Student
2. XYZ789 Rahul Admin

1. The server creates a unique ID called a Session ID and stores the user's information against that ID.
2. The browser mainly gets: ABC123

CLIENT STORES SESSION ID

1. The server sends the Session ID to the browser.
   Server -----> Session ID ------> ABC123

2. Usually, the browser stores the session ID in a **cookie**.

CLIENT SENDS SESSION ID IN FUTURE REQUESTS

1. User creates requests for profile
   GET /profile
   Cookie: sessionId=ABC123

2. The browser is not sending the username and password again.
3. It Only Sends
   Session ID = ABC123

SERVER VALIDATES SESSION ID

1. The server receives
   ABC123
2. Now it checks its session store.
   ABC123 → Mayur → Student
3. Server remembers the user.

SERVER SENDS RESPONSE

Session valid ------> Allow Request -----> Return users profile

User first logs in with his username and password. The server checks his credentials. If they are correct, the server creates a session and stores Mayur's information on the server. The server gives the browser a Session ID, for example ABC123, usually inside a cookie. Later, when User requests his profile, the browser automatically sends ABC123. The server looks up ABC123 in its session store and finds that it belongs to User. If the session is valid, the server returns his profile. If it is invalid, the server returns 401 Unauthorized."
